Lexical Analyzer Design Report

1. Design Decisions

The goal of this project was to implement a lexical analyzer (lexer) that reads source code from a file and converts it into a sequence of tokens. Each token represents a meaningful unit in the code, such as keywords, identifiers, operators, literals, and separators.

Key design decisions include:

1. **Token Types**: The lexer recognizes the following token types:
    - **KEYWORD**: Reserved words like int, float, if, else, while, return.
    - **IDENTIFIER**: Variable names or function names.
    - **OPERATOR**: Symbols that represent operations, such as +, -, *, =, ==, etc.
    - **LITERAL**: Numbers (both integer and floating-point).
    - **SEPARATOR**: Symbols that separate different parts of the code, such as parentheses (), braces {}, semicolons ;, commas ,, etc.
    - **WHITESPACE**: Spaces and tabs, which are ignored during tokenization.
    - **COMMENT**: Lines starting with //, ignored during tokenization.
    - **INVALID**: Any unrecognized token or character that doesn't match one of the specified categories.
2. **Input Handling**: The lexer takes an input file, input_program.txt, which contains the source code. It reads the entire file into memory line by line using readlines() to process each line individually.
3. **Tokenization Mechanism**: A **regular expression-based approach** was used to identify tokens in the source code. This approach provides an efficient way to match patterns for keywords, identifiers, operators, and literals.
    - Regular expressions for each token type were defined in the token_specification list.
    - A single combined regular expression was created using |, which matches any of the token types in the list.
    - The re.finditer method was used to find all matches in a line, and each match is processed to determine its type.
4. **Line Number Tracking**: The lexer tracks the line number of each token in the source code to provide helpful error messages when invalid tokens are found.

2. Implementation Details

- **Token Class**: The Token class encapsulates a token's type, value (lexeme), and line number. It also provides a __str__ method to display token information in a readable format.

- **Lexer Class**: The Lexer class is the core of the lexical analyzer:
    - **Constructor**: The constructor takes a filename, reads the source code, and initializes the list of keywords, operators, and separators. The line number tracker is initialized to 1.
    - **tokenize Method**: This method reads the source code line by line and calls the tokenize_line method for each line. It appends recognized tokens to the tokens list.
    - **tokenize_line Method**: This method defines the regular expressions for each token type. It iterates over the matches using re.finditer and processes them. Invalid tokens and illegal identifiers are flagged and reported as errors.
    - **is_valid_identifier Method**: This method checks that identifiers don't start with a digit, as that would make them illegal in many programming languages.

3. Error Handling Mechanisms

Error handling is a critical component of the lexer to ensure that invalid tokens and illegal identifiers are caught. Here's an overview of the error handling implemented:

1. **Unrecognized Tokens**: If a token is encountered that doesn't match any of the defined token patterns, it is considered an "unrecognized token". These tokens are handled by the INVALID regular expression pattern, which matches any character that doesn't fit into one of the valid categories.

   Example:

   Error: Unrecognized token '@4;' on line 14

2. **Illegal Identifiers**: Identifiers that start with a digit (e.g., 1er, 2abc) are flagged as illegal identifiers. This is handled by the is_valid_identifier method, which checks if the first character of an identifier is a digit. If it is, an error message is printed.

   Example:

   Error: Unrecognized token '1er;' on line 12

   Error: Unrecognized token '2tg;' on line 13

3. **Whitespace and Comments**: The lexer ignores whitespace and comments. These tokens are matched by the WHITESPACE and COMMENT regular expressions, but no tokens are created for them. They do not trigger any errors but are simply skipped over.
4. **Line Number Reporting**: Each error includes the line number where the invalid token or identifier was found, making it easier for the user to locate and fix the issue in the source code

4. Test Cases

The following test cases were used to validate the lexer:

1. **Valid Program Example**: Input:

```
input_program.txt
int main() {
    int x = 10;
    float y = 20.5;
    if (x > y) {
        x = x + 1;
    }

    // This is a comment
    else {
        y = y - 2.5;
    }
    int 1er;
    int 2tg;
    int c = 3@4;
}
```

Expected Output:

```
Error: Unrecognized token '1er;' on line 12
Error: Unrecognized token '2tg;' on line 13
Error: Unrecognized token '@4;' on line 14
Token(KEYWORD, int, Line 1)
Token(IDENTIFIER, main, Line 1)
Token(SEPARATOR, (, Line 1)
Token(SEPARATOR, ), Line 1)
Token(SEPARATOR, {, Line 1)
Token(KEYWORD, int, Line 2)
Token(IDENTIFIER, x, Line 2)
Token(OPERATOR, =, Line 2)
Token(LITERAL, 10, Line 2)
Token(SEPARATOR, ;, Line 2)
Token(KEYWORD, float, Line 3)
Token(IDENTIFIER, y, Line 3)
Token(OPERATOR, =, Line 3)
Token(LITERAL, 20.5, Line 3)
Token(SEPARATOR, ;, Line 3)
Token(KEYWORD, if, Line 4)
Token(SEPARATOR, (, Line 4)
Token(IDENTIFIER, x, Line 4)
Token(OPERATOR, >, Line 4)
Token(IDENTIFIER, y, Line 4)
Token(SEPARATOR, ), Line 4)
Token(SEPARATOR, {, Line 4)
Token(IDENTIFIER, x, Line 5)
Token(OPERATOR, =, Line 5)
Token(IDENTIFIER, x, Line 5)
Token(OPERATOR, +, Line 5)
Token(LITERAL, 1, Line 5)
Token(SEPARATOR, ;, Line 5)
Token(SEPARATOR, }, Line 6)
```

```
Token(OPERATOR, /, Line 8)
Token(OPERATOR, /, Line 8)
Token(IDENTIFIER, This, Line 8)
Token(IDENTIFIER, is, Line 8)
Token(IDENTIFIER, a, Line 8)
Token(IDENTIFIER, comment, Line 8)
Token(KEYWORD, else, Line 9)
Token(SEPARATOR, {, Line 9)
Token(IDENTIFIER, y, Line 10)
Token(OPERATOR, =, Line 10)
Token(IDENTIFIER, y, Line 10)
Token(OPERATOR, -, Line 10)
Token(LITERAL, 2.5, Line 10)
Token(SEPARATOR, ;, Line 10)
Token(SEPARATOR, }, Line 11)
Token(KEYWORD, int, Line 12)
Token(IDENTIFIER, er, Line 12)
Token(SEPARATOR, ;, Line 12)
Token(KEYWORD, int, Line 13)
Token(IDENTIFIER, tg, Line 13)
Token(SEPARATOR, ;, Line 13)
Token(KEYWORD, int, Line 14)
Token(IDENTIFIER, c, Line 14)
Token(OPERATOR, =, Line 14)
Token(LITERAL, 3, Line 14)
Token(LITERAL, 4, Line 14)
Token(SEPARATOR, ;, Line 14)
Token(SEPARATOR, }, Line 15)
```